1. Contexte et périmètre

Correction après vérification V4, 12 novembre 2026. Tests techniques et résultats de développement. Revue humaine indépendante non terminée.

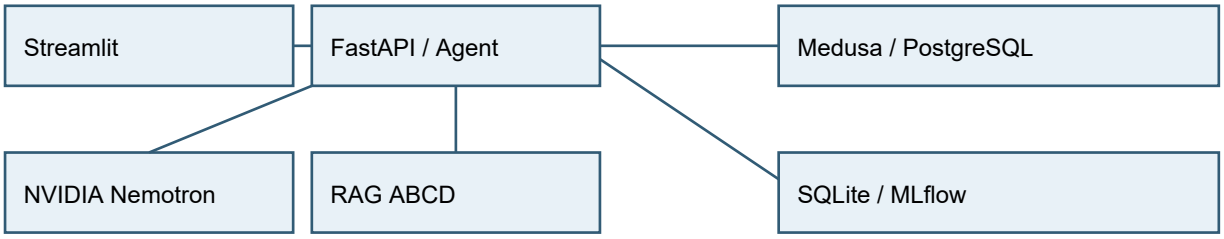
CommerceOps AI relie une conversation en français à une documentation de support et à une boutique Medusa. Un client recherche un produit, prépare une approbation, confirme explicitement la création d’un panier et consulte son contenu réel. Les questions de procédure suivent un chemin documentaire avec références. Les données transactionnelles sont obtenues sous l’identité du client authentifié.

Un client doit comprendre les possibilités décrites par une source et savoir ce que la boutique a réellement effectué. Une procédure ABCD peut éclairer une situation sans établir une règle contractuelle. Une réponse du modèle ne prouve pas la réussite d’une opération. Ces distinctions structurent le projet et les messages présentés au client.

La version décrite conserve FastAPI, Streamlit, NVIDIA Nemotron, les composants RAG, SQLite, MLflow, Medusa et PostgreSQL. Les corrections portent sur les contrôles et leurs tests. Le projet demeure une démonstration universitaire locale. Les remboursements financiers autonomes restent interdits et les retours dépendent d’une activation marchande explicite.

L’audit initial avait relevé une reproductibilité insuffisante, une possible dérive de variante, des avis de sécurité et des livrables obsolètes. La version corrigée ajoute une photographie commerciale approuvée, un traitement des écritures incertaines et des fixtures documentaires isolées. Les 163 tests locaux et les 163 tests Python 3.12 propres réussissent. Ces vérifications techniques ne mesurent pas une généralisation indépendante.

2. Architecture et données



Streamlit fournit la connexion, la conversation, les cartes produit et panier, ainsi que les boutons de confirmation. FastAPI expose les sessions, les recherches et les transitions d’approbation. Le backend authentifie les demandes métier avec Medusa avant d’accéder aux références persistées. Les interfaces habituelles utilisent les ports locaux 8501 et 8001.

Medusa conserve les produits et paniers dans PostgreSQL. Le connecteur utilise la Store API et une clé publishable. Le JWT du client accompagne les requêtes protégées. Les vérifications de propriété s’appliquent dans le connecteur et les middlewares Medusa. PostgreSQL et Redis sont conteneurisés avec des ports liés à la boucle locale. Le bus et le verrouillage de cette démonstration restent en mémoire.

SQLite conserve les tickets, les approbations, les sessions et les événements de CommerceOps. Une session opaque appartient à un client précis. L’historique contient des messages canoniques et des références autorisées. Les données reconnues comme sensibles sont filtrées. Cette détection reste heuristique.

NVIDIA sélectionne les identifiants de faits documentaires autorisés et propose des appels structurés lorsque le chemin agentique l’exige. Le modèle configuré est nvidia/nemotron-3.5-lightning-30b-a3b. Le backend valide les schémas et les résultats. MLflow conserve des traces expurgées avec outils, durées et statuts. Les arguments bruts, les JWT et les adresses sont exclus de ce canal.

3. Ingestion et recherche documentaire

L’ingestion accepte les objets ou listes JSON documentaires, le texte et Markdown avec provenance explicite. Chaque document possède un titre, une catégorie, un texte et une URI de source. Les formats texte utilisent un fichier annexe ou un front matter. Les entrées sans texte exploitable ou provenance valide sont refusées.

Le texte indexé inclut titre, catégorie et provenance. Les chunks conservent leurs identifiants et extraits. L’empreinte du document permet de vérifier le contexte réutilisé. L’index actif configuré compte 55 documents et 217 chunks. Le fichier historique par défaut peut représenter une autre génération.

La reconstruction utilise un verrou, un fichier temporaire et un remplacement atomique. Une construction en échec conserve la génération précédente. La clé de cache dépend du contenu chargé. Le manifeste dense relie embeddings et chunks à une génération commune afin d’éviter les mélanges d’index.

Le backend courant est lexical. Il classe les documents par scores TF-IDF du corps et du titre, avec un pont de termes français et anglais. Les références des chunks restent accessibles. Les baselines lexicales et denses, la fusion hybride et le reranking sont conservés. Leurs performances indépendantes n’ont pas été remesurées dans cette correction.

Les tests construisent leur propre index à partir de quatre petits ensembles d’extraits publics ABCD. La licence MIT et la provenance accompagnent ces fixtures. Le répertoire temporaire est nettoyé à la fin des tests. Le corpus et l’index personnel du développeur sont absents du conteneur de reproduction.

4. Synthèse et fidélité aux sources

La chaîne documentaire distingue récupération, pertinence, faits soutenus et présentation. Le backend reconnaît un ensemble borné de faits associés à des extraits exacts. NVIDIA sélectionne leurs identifiants dans une liste autorisée. Une sortie invalide déclenche un repli sur les seuls faits vérifiés. La prose fournisseur ne devient pas une preuve métier.

Chaque unité de réponse conserve les faits, les extraits et les chunks d’origine. Les conditions restent liées aux actions. Pour un retour dû à la couleur, la préparation de l’étiquette exige le passage décrivant l’acceptation du retour. Les modes postal, magasin et centre sont attribués à ABCD. L’éligibilité et les modes réellement proposés par la boutique restent à vérifier.

La demande « résume cette procédure en trois étapes » conserve son format et son contexte documentaire. Le backend produit les unités numérotées étayées. Lorsqu’une condition ou une étape manque, il nomme cette absence. Les versions courtes et détaillées gardent la citation et la portée de la source.

Le dialogue public réel a été rejoué. Sa réponse courte passe de 687 à 577 caractères, citation comprise. Ce comptage décrit la longueur observée. Les réponses intégrales figurent dans rag-before-after.json. Les autres sous-flux emploient désormais des verbes concrets comme « se déconnecter puis se reconnecter » lorsque l’extrait le dit.

Les politiques officielles absentes et les procédures inconnues déclenchent une abstention. Les traductions et règles de pertinence demeurent bornées. Une citation exacte ne suffit pas à prouver le sens de toute affirmation. La fidélité humaine indépendante reste non évaluée.

5. Mémoire et interface client

Une conversation authentifiée reçoit un identifiant opaque lié au client Medusa. Le backend conserve le produit, la variante, la quantité, l’intention et les actions en attente. Cet état permet d’interpréter « oui », « non », « celui-là » et une précision de taille. Les visiteurs non authentifiés ne disposent pas de cette mémoire persistante.

Un « oui » prépare une approbation avec des arguments revérifiés. L’interface exige ensuite une case explicite et le bouton Confirmer. Un changement de produit ou d’avis annule les propositions en attente. Une annulation locale ne déclenche pas une annulation de commande dans Medusa.

Le suivi documentaire réexamine la source et son empreinte. Un échec de recherche est mémorisé pour éviter de résumer une procédure inexistante. La consultation de panier utilise une référence liée à la session et revérifie la propriété. Une nouvelle conversation efface ce lien local. Un changement d’identité efface le contexte d’interface précédent. Les corrections retiennent la dernière taille explicite et les nombres français. Une annulation naturelle modifie l’approbation persistante. Les demandes déjà en cours ou terminées reçoivent un message décrivant leur état réel. L’ancienne notification de réussite est effacée au nouvel échange.

Streamlit affiche les produits, variantes, quantités et vrais statuts métier. Après création confirmée, la carte panier présente les lignes et montants relus dans Medusa. Le stock reste explicitement non vérifié. La présence d’une variante au catalogue ne démontre pas sa disponibilité. Un prix absent reste inconnu jusqu’à sa lecture dans la boutique.

Le JSON et les identifiants techniques sont réservés au panneau développeur facultatif. AppTest vérifie les composants et les transitions avec le vrai backend HTTP. Ce contrôle structurel est distinct d’une inspection visuelle du navigateur, qui reste en attente.

6. Approbations et écritures incertaines

La préparation de panier conserve une photographie commerciale côté serveur. Elle contient les identifiants, libellés, options et attributs de variante, ainsi que prix et devise lorsqu'ils sont fournis. Les informations affichées proviennent de cette photographie. Les approbations anciennes qui en sont dépourvues exigent une nouvelle préparation.

La confirmation authentifie le client et vérifie le lien à sa session. Une transition SQLite conditionnelle permet à un seul appel concurrent de revendiquer l'approbation. Le catalogue est ensuite relu avant l'écriture. Une disparition ou une différence de produit, taille, option, SKU, prix ou devise invalide l'action. Le client doit obtenir un nouveau récapitulatif.

Le checkout conserve le contrôle de propriété et compare le contenu et les montants du panier approuvé. Les retours conservent leurs vérifications marchandes. Cette correction ne teste aucune nouvelle commande réelle ni remboursement. La lecture de catalogue et l'écriture restent deux appels distincts. Aucune réservation de stock ni atomicité interservices n'est promise.

Créer le panier et ajouter l'article demandent deux écritures. Si la réponse d'ajout est perdue, le connecteur consulte le panier propriétaire. La présence exacte de la ligne attendue permet de confirmer l'ajout. Sinon, l'approbation passe à needs_review. Le message indique le panier connu ou la possibilité d'une création sans réponse exploitable.

Un panier partiel connu reste consultable. L'interface conseille la consultation ou un conseiller. Aucun POST n'est répété automatiquement. Les tests couvrent réponses perdues, panier vide, concurrence, double confirmation et accès inter-clients.

La procédure opérateur pour un résultat needs_review est décrite dans docs/MEDUSA_UNCERTAIN_WRITES.md. Elle impose une vérification en lecture seule et une nouvelle confirmation du client avant toute autre action ; elle ne prétend pas rendre atomiques les deux écritures Medusa.

7. Dépendances et sécurité résiduelle

Pip est verrouillé à 26.3. L'audit du lock Windows trouve quatre avis Chroma distincts sur chromadb==1.5.9 (cinq entrées de sortie, dont un identifiant répété) et aucune version PyPI corrigée disponible au 12 novembre 2026. Chroma n'est présent que dans requirements-mlops.txt et le lock Windows MLOps ; le lock Linux Python 3.12 et l'installation de base n'en dépendent pas. Le backend courant est lexical. Le code dense utilise chromadb.PersistentClient local et ne démarre pas de serveur HTTP Chroma. trust_remote_code=False concerne SentenceTransformers, pas l'API Chroma. Les services Chroma HTTP/RBAC et toute exposition réseau restent interdits jusqu'à publication d'un correctif et nouvelle revue.

Le groupe JavaScript utilise pnpm 10.11.1, déclaré par le monorepo. Next.js et son profil ESLint passent à 16.4.1, Zod à 4.6.6. Les correctifs transitifs sont bornés aux versions majeures existantes. Le lockfile est régénéré par pnpm. Les imports et le lint Medusa passent. Le build du storefront passe avec la clé publishable locale injectée en mémoire.

Les sept avis de l'audit V2 sont tracés individuellement dans le rapport V3 : quatre Chroma restent signalés mais leurs préconditions HTTP/RBAC sont absentes du chemin exécuté ; UUID 11.1.1 et minimatch 11.0.1 ont été corrigés ; deepmerge@4.3.1 reste signalé par npm. Body-parser 1.20.9 était un correctif séparé.

Le paquet deepmerge est présent dans le graphe du lock Medusa via Jest et via le graphe de dépendances de @medusajs/medusa → SendGrid. Le provider SendGrid n'est pas configuré dans medusa-config.ts, mais l'audit --prod confirme le chemin transitif. Aucun correctif upstream n'est publié et le registre ne présente pas de version supérieure à 4.3.1. Un test avec une source JSON __proto__ a reproduit la modification du prototype de l'objet fusionné. Un patch pnpm local rejette cette clé dans les entrées CommonJS et UMD ; le test test:security exerce le paquet résolu depuis le chemin SendGrid. pnpm audit continue de signaler l'avis faible sur la version upstream et son code de sortie reste non nul : l'avis n'est ni ignoré ni masqué.

Le scan ne remplace pas un test d'intrusion. Les secrets restent hors des preuves. La révocation de l'ancienne clé NVIDIA repose sur la confirmation de l'utilisateur et n'a pas été contrôlée dans le portail émetteur.

8. Validation et résultats réels

La suite locale utilise Python 3.14 et compte 163 tests réussis. Les cas supplémentaires vérifient les caractéristiques approuvées, la disparition de variante, les prix, la concurrence, les anciennes approbations, les écritures perdues et les diagnostics fournisseur. Les régressions RAG, sécurité, mémoire et interface restent incluses.

La reproduction propre installe requirements-linux-py312.lock.txt dans un conteneur Python 3.12 neuf. Les sources sont montées en lecture seule. Corpus personnel, bases, clés, index actif et packages du poste sont absents. Les fixtures construisent l'index pendant les tests. Cette exécution compte aussi 163 réussites. L'audit de ce lock Linux ne trouve aucun avis connu. GitHub Actions utilise le même fichier de dépendances.

Vérification	Résultat courant
Pytest local et Python 3.12 propre	163 réussites chacun
Ruff au profil CI (E4,E7,E9,F63,F7,F82)	Réussi localement et dans le conteneur Python 3.12
Gates historique et développement	Réussis avec les résultats archivés
Hygiène secrets et pip check	Réussis localement
Audit pip Windows / Linux	Quatre avis Chroma distincts ; aucun avis dans le lock Linux
Installation pnpm gelée, test de patch et lint Medusa	Réussis
Audit npm	Un avis faible deepmerge reste signalé ; le patch local est testé
Ruff sans profil CI, mesure historique V3	55 diagnostics hors profil CI : 41 I001, 6 F401, 3 F841, 4 S110, 1 B023

Le parcours de correction conversationnelle utilise les fixtures de démonstration autorisées. L'approbation annulée refuse une confirmation ultérieure avec HTTP 409. Après changement de taille et de quantité, un panier de sweatshirt L, quantité 2, est explicitement confirmé puis relu dans Medusa. Une création de panier est effectuée pendant cette campagne. Aucune commande ni remboursement n'est demandé.

Les refus inter-clients, l'absence d'authentification, l'annulation et la confirmation unique sont vérifiés. Les listes de commandes sont identiques avant et après. Le harnais ne demande aucun remboursement. Les rapports gardent les assertions et les résultats sans JWT ni identifiants clients.

9. Reproduction et préparation Git

Les commandes principales sont python -m scripts.verify_pre_push_v2 local et python -m scripts.verify_pre_push_v2 clean. Le second mode exige Docker. Il crée une copie temporaire des sources et la détruit après le conteneur. Les preuves des campagnes précédentes restent dans reports/pre-push/v2/; les preuves conversationnelles et la validation PDF courantes sont dans reports/pre-push/conversation-final/. Les données privées sont extérieures à ces copies.

Le starter Medusa et ses étapes de configuration sont documentés. Les manifests et le lockfile corrigés figurent dans integrations/medusa/dependencies/. Le README indique leurs destinations. Les secrets PostgreSQL, JWT et cookies sont générés localement. Les empreintes des middlewares de propriété sont contrôlées avant et après correction.

L’inventaire Git vérifie les fichiers nécessaires, dont app/documentary_format.py, app/catalog.py et les fixtures. Il distingue les deux dépôts locaux et leurs candidats à publication. Les bases, secrets, credentials et fichiers temporaires sont exclus de la préparation. Cette mission ne réalise ni commit ni push.

Les preuves initiales sont conservées pour la comparaison avant/après. L’archive du premier audit décrit son état historique. Le rapport V3 distingue les sources et résultats courants. La liste de préparation Git de V3 exclut les secrets, bases, caches et fichiers temporaires. Aucun commit ni push n’a été effectué.

La reproduction Python et les parcours couvrent le périmètre observé. La reconstruction complète de PostgreSQL, l’import intégral et la charge sur un second poste restent à valider. Le storefront optionnel a été compilé. L’interface client exercée par AppTest demeure Streamlit.

10. Évaluation indépendante et limites

Les anomalies fonctionnelles ont reçu des corrections ciblées et des tests. La confirmation refuse les différences commerciales. Les tests utilisent un index autonome. Les écritures ambiguës restent bloquées contre la répétition et consultables lorsqu'un panier est connu. Les fallbacks exposent des catégories bornées sans contenu d'erreur fournisseur brut.

Les catégories comprennent fournisseur absent, timeout, indisponibilité, erreur HTTP, payload invalide et fait non autorisé. Elles apparaissent dans les diagnostics et événements techniques. Le client garde une réponse liée à la source et aux limites. Les formats distincts du dialogue public ne garantissent pas la qualité sur toute formulation française.

Le jeu final gelé de 16 scénarios et son formulaire conservent leurs empreintes. Ils n'ont pas servi à ajuster cette correction. La revue humaine reste non approuvée. Les résultats de développement gardent leur étiquette. Les métriques indépendantes de fidélité et de réussite de tâche restent non évaluées.

Le protocole demande de confronter intentions, sources, abstentions et affirmations aux passages originaux. Une citation exacte établit une correspondance textuelle. Le relecteur doit aussi contrôler le sens, les conditions, la portée commerciale et le français. La mesure indépendante intervient après cette revue et un protocole fixé à l'avance.

Le jeu final gelé de 16 scénarios n'a pas été modifié. La revue humaine indépendante, le contrôle visuel du navigateur, les performances en charge et l'atomicité interservices restent non évalués. Le patch local de deepmerge exige une surveillance et un remplacement par une version upstream corrigée lorsqu'elle sera publiée. Les quatre avis Chroma demeurent visibles ; le blocage de leur activation/exposition est une condition de déploiement. La pagination, les figures, les tableaux et les rendus du PDF sont contrôlés après génération.

Références techniques consultées : jeu documentaire ABCD (github.com/asappresearch/abcd), documentation Medusa Store API (docs.medusajs.com/api/store), avis Chroma GHSA-f4j7-r4q5-qw2c, GHSA-36p7-vc44-83pf, GHSA-xph7-9rjv-w5fr et GHSA-2wm9-hf6c-p5cr, et avis deepmerge GHSA-gcqx-w48m-w2h4 (github.com/advisories/).